29th July 2026

Lab 9: File Uploads & External APIs – “SendIt” Document Management API

1. Must have completed lessons 1 to 8
2. note carefully how each step compares with the lab 8’s steps, and even how it compares with similar steps of previous labs.

Lab Scenario: SendIt - A Document Management & Enrichment API

The Problem

A Nyeri courier company, "SendIt," wants to digitize their document management process. They currently handle thousands of physical documents daily (waybills, invoices, customs forms). Their challenges include:

1. No digital document storage – everything is on paper
2. No way to enrich documents – they want to automatically add weather conditions at pickup/delivery locations
3. Manual tracking – no way to know if a document was uploaded correctly
4. No validation – staff upload invalid files (wrong type, too large)
5. No external data integration – they need to pull weather data for delivery locations

Your Task: Build a document management API that:

- Uploads documents (PDFs, images) with validation
- Enriches documents with external data (weather API)
- Tracks document status (uploaded, processing, enriched, failed)
- Provides secure access based on user roles

Step 2: Create the File Structure

```
sendit-api/
├── main.py
├── models/
│   ├── __init__.py
│   ├── user.py
│   └── document.py
├── database/
│   ├── __init__.py
│   └── session.py
├── services/
│   ├── __init__.py
│   └── weather.py
├── uploads/                # Directory for uploaded files
├── auth.py
├── seeds.py
├── .env
├── docker-compose.yml
```

└── alembic.ini

compare this file directory with the previous ones.

Step 3: Set Up PostgreSQL and Create Uploads Directory

```
# docker-compose.yml
cat > docker-compose.yml << 'EOF'
services:
  db:
    image: postgres:16
    container_name: sendit_db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: sendit_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
EOF

docker compose up -d

# Create uploads directory (as done in Lesson 4 for static files)
mkdir -p uploads
```

Step 4: Configure Environment

```
# .env
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/sendit_db
SECRET_KEY=your-super-secret-key-change-in-production
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30

# Weather API (use a free one or mock for development)
WEATHER_API_KEY=your-api-key
WEATHER_API_URL=https://api.open-meteo.com/v1/forecast

# File upload limits (as discussed in Lesson 9)
MAX_UPLOAD_SIZE=5242880  # 5 MB
```

ALLOWED_EXTENSIONS=.pdf,.jpg,.jpeg,.png,.docx

Part 2: Models and Database

Step 5: Create the User Model

```
# models/user.py
from sqlmodel import SQLModel, Field
from datetime import datetime
from typing import Optional, List
from models.document import Document

class User(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    username: str = Field(unique=True, index=True, min_length=3, max_length=50)
    email: str = Field(unique=True, index=True)
    hashed_password: str
    full_name: str = Field(min_length=2, max_length=100)
    role: str = Field(default="staff")  # "admin", "manager", "staff"
    is_active: bool = Field(default=True)
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)
    last_login: Optional[datetime] = None

    # Relationship: documents uploaded by this user
    documents: List["Document"] = Relationship(back_populates="uploader")

class UserCreate(SQLModel):
    username: str = Field(min_length=3, max_length=50)
    email: str
    password: str = Field(min_length=8)
    full_name: str = Field(min_length=2, max_length=100)
    role: str = Field(default="staff")

class UserLogin(SQLModel):
    username: str
    password: str

class UserResponse(SQLModel):
    id: int
    username: str
    email: str
    full_name: str
    role: str
```

```
    is_active: bool
    created_at: datetime
```

Step 6: Create the Document Model

```
# models/document.py
from sqlmodel import SQLModel, Field, Relationship
from datetime import datetime
from typing import Optional

class Document(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    filename: str
    original_filename: str
    file_size: int  # in bytes
    file_type: str  # MIME type
    status: str = Field(default="uploaded")  # "uploaded", "processing", "enriched", "failed"

    # Location data
    city: str = Field(index=True)
    country: str = Field(default="Kenya")

    # Weather data (from external API)
    weather_data: Optional[str] = Field(default=None)  # JSON string
    weather_fetched_at: Optional[datetime] = None

    # Metadata
    description: Optional[str] = None
    uploader_id: int = Field(foreign_key="user.id")
    uploader: "User" = Relationship(back_populates="documents")
    uploaded_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)

    # File path on server
    file_path: str

class DocumentCreate(SQLModel):
    city: str = Field(min_length=2, max_length=100)
    country: str = Field(default="Kenya", min_length=2, max_length=100)
    description: Optional[str] = None

class DocumentUpdate(SQLModel):
    city: Optional[str] = Field(None, min_length=2, max_length=100)
    country: Optional[str] = Field(None, min_length=2, max_length=100)
```

```
    description: Optional[str] = None
```

Part 3: Authentication and Authorization

Step 7: Implement Authentication (Reuse from Lab 8)

```
# auth.py - Reuse from Lab 8 with same role structure
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from fastapi import HTTPException, Depends, status
from fastapi.security import OAuth2PasswordBearer
from sqlmodel import Session, select
import os

from database.session import get_session
from models.user import User

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

SECRET_KEY = os.getenv("SECRET_KEY")
ALGORITHM = os.getenv("ALGORITHM", "HS256")
ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES",
"30"))

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def decode_access_token(token: str) -> dict:
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    except JWTError:
        raise HTTPException(401, "Invalid or expired token")
```

```
def get_current_user(
    token: str = Depends(oauth2_scheme),
    session: Session = Depends(get_session)
) -> User:
    payload = decode_access_token(token)
    username = payload.get("sub")
    if not username:
        raise HTTPException(401, "Invalid token")

    user = session.exec(select(User).where(User.username == username)).first()
    if not user:
        raise HTTPException(401, "User not found")

    if not user.is_active:
        raise HTTPException(403, "User is inactive")

    return user

def get_current_admin(current_user: User = Depends(get_current_user)) -> User:
    if current_user.role != "admin":
        raise HTTPException(403, "Admin access required")
    return current_user

def get_current_manager(current_user: User = Depends(get_current_user)) -> User:
    if current_user.role not in ["admin", "manager"]:
        raise HTTPException(403, "Manager or admin access required")
    return current_user
```

Part 4: File Upload Endpoints

Step 8: Implement File Upload with Validation

```
# main.py
from fastapi import FastAPI, File, UploadFile, HTTPException, Depends, status, Request, Form
from fastapi.responses import JSONResponse
from fastapi.security import OAuth2PasswordRequestForm
from sqlmodel import Session, select
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
from datetime import datetime
import os
import shutil
```

```
import aiofiles
import json
from typing import Optional

from database.session import get_session
from models.user import User, UserCreate, UserResponse
from models.document import Document, DocumentCreate, DocumentUpdate
from auth import (
    hash_password, verify_password, create_access_token,
    get_current_user, get_current_admin, get_current_manager
)
from services.weather import get_weather

app = FastAPI(title="SendIt API", version="1.0.0")

# ============================================================
# CONFIGURATION
# ============================================================

UPLOAD_DIR = "uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

MAX_FILE_SIZE = int(os.getenv("MAX_UPLOAD_SIZE", 5 * 1024 * 1024))  # 5 MB
ALLOWED_EXTENSIONS = [".pdf", ".jpg", ".jpeg", ".png", ".docx"]

# ============================================================
# RATE LIMITING
# ============================================================

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

# ============================================================
# AUTHENTICATION ENDPOINTS (Reuse from Lab 8)
# ============================================================

# ... (register, login endpoints from previous lab)

# ============================================================
# FILE UPLOAD ENDPOINTS
# ============================================================
```

```
def validate_file(file: UploadFile) -> tuple:
    """Validate file size and type. Returns (is_valid, error_message)"""
    # Check file extension
    file_extension = os.path.splitext(file.filename)[1].lower()
    if file_extension not in ALLOWED_EXTENSIONS:
        return False, f"File type not allowed. Allowed types: {', '.join(ALLOWED_EXTENSIONS)}"

    # Check file size (must read to know size)
    # We'll check in the endpoint by reading the file
    return True, ""

@app.post("/documents/upload")
@limiter.limit("10/hour")
async def upload_document(
    request: Request,
    file: UploadFile = File(...),
    city: str = Form(...),
    description: Optional[str] = Form(None),
    country: str = Form("Kenya"),
    current_user: User = Depends(get_current_user),
    session: Session = Depends(get_session)
):
    """
    Upload a document with validation.
    Enriches with weather data for the specified city.
    """
    # 1. Validate file extension
    file_extension = os.path.splitext(file.filename)[1].lower()
    if file_extension not in ALLOWED_EXTENSIONS:
        raise HTTPException(400, f"File type not allowed. Allowed: {',
'.join(ALLOWED_EXTENSIONS)}")

    # 2. Read and validate file size
    contents = await file.read()
    file_size = len(contents)
    if file_size > MAX_FILE_SIZE:
        raise HTTPException(400, f"File too large. Max size: {MAX_FILE_SIZE // (1024*1024)} MB")

    # 3. Generate safe filename
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    safe_filename = f"{timestamp}_{current_user.id}_{file.filename.replace(' ', '_')}"
```

```
file_path = os.path.join(UPLOAD_DIR, safe_filename)

# 4. Save file asynchronously
async with aiofiles.open(file_path, 'wb') as out_file:
    await out_file.write(contents)

# 5. Create document record
document = Document(
    filename=safe_filename,
    original_filename=file.filename,
    file_size=file_size,
    file_type=file.content_type or "application/octet-stream",
    city=city,
    country=country,
    description=description,
    uploader_id=current_user.id,
    file_path=file_path,
    status="processing"
)
session.add(document)
session.commit()
session.refresh(document)

# 6. Enrich with weather data (external API call)
try:
    weather_data = await get_weather(city, country)
    if weather_data:
        document.weather_data = json.dumps(weather_data)
        document.weather_fetched_at = datetime.utcnow()
        document.status = "enriched"
        session.commit()
except Exception as e:
    # Log error but don't fail the upload
    print(f"Weather API error: {e}")
    document.status = "uploaded"
    session.commit()

return {
    "message": "Document uploaded successfully",
    "document_id": document.id,
    "filename": document.original_filename,
    "status": document.status
```

```
    }

@app.get("/documents")
@limiter.limit("30/minute")
def list_documents(
    request: Request,
    status: Optional[str] = None,
    city: Optional[str] = None,
    current_user: User = Depends(get_current_user),
    session: Session = Depends(get_session)
):
    """List all documents with optional filters."""
    query = select(Document)

    # Managers and admins see all documents; staff see only their own
    if current_user.role not in ["admin", "manager"]:
        query = query.where(Document.uploader_id == current_user.id)

    if status:
        query = query.where(Document.status == status)
    if city:
        query = query.where(Document.city == city)

    return session.exec(query).all()

@app.get("/documents/{document_id}")
@limiter.limit("30/minute")
def get_document(
    request: Request,
    document_id: int,
    current_user: User = Depends(get_current_user),
    session: Session = Depends(get_session)
):
    """Get a specific document."""
    document = session.get(Document, document_id)
    if not document:
        raise HTTPException(404, "Document not found")

    # Staff can only view their own documents
    if current_user.role not in ["admin", "manager"] and document.uploader_id != current_user.id:
        raise HTTPException(403, "Access denied")
```

```
    return document

@app.delete("/documents/{document_id}")
def delete_document(
    document_id: int,
    current_user: User = Depends(get_current_manager),
    session: Session = Depends(get_session)
):
    """Delete a document (managers and admins only)."""
    document = session.get(Document, document_id)
    if not document:
        raise HTTPException(404, "Document not found")

    # Delete physical file
    if os.path.exists(document.file_path):
        os.remove(document.file_path)

    session.delete(document)
    session.commit()
    return {"message": "Document deleted successfully"}
```

Part 5: External API Integration

Step 9: Implement Weather Service

```
# services/weather.py
import httpx
import os
from datetime import datetime
from typing import Optional, Dict

# Free weather API (Open-Meteo) - no API key required for basic use
WEATHER_API_URL = os.getenv("WEATHER_API_URL",
"https://api.open-meteo.com/v1/forecast")
GEOCODING_API_URL = "https://geocoding-api.open-meteo.com/v1/search"

async def get_coordinates(city: str, country: str = "Kenya") -> Optional[tuple]:
    """Get latitude and longitude for a city using Open-Meteo geocoding."""
    async with httpx.AsyncClient() as client:
        try:
            response = await client.get(
                GEOCODING_API_URL,
                params={
                    "name": city,
```

```
                "count": 1,
                "language": "en",
                "format": "json"
            },
            timeout=10.0
        )
        response.raise_for_status()
        data = response.json()

        if data.get("results"):
            result = data["results"][0]
            return (result["latitude"], result["longitude"])
    except Exception as e:
        print(f"Geocoding error: {e}")
return None

async def get_weather(city: str, country: str = "Kenya") -> Optional[Dict]:
    """
    Fetch current weather for a city using Open-Meteo API.
    """
    coordinates = await get_coordinates(city, country)
    if not coordinates:
        return {"error": "Could not find city coordinates"}

    lat, lon = coordinates

    async with httpx.AsyncClient() as client:
        try:
            response = await client.get(
                WEATHER_API_URL,
                params={
                    "latitude": lat,
                    "longitude": lon,
                    "current_weather": True,
                    "temperature_unit": "celsius",
                    "timezone": "Africa/Nairobi"
                },
                timeout=10.0
            )
            response.raise_for_status()
            data = response.json()
```

```
            current = data.get("current_weather", {})
            return {
                "city": city,
                "country": country,
                "temperature": current.get("temperature"),
                "windspeed": current.get("windspeed"),
                "weathercode": current.get("weathercode"),
                "time": current.get("time"),
                "source": "Open-Meteo"
            }
        except httpx.TimeoutException:
            print(f"Weather API timeout for {city}")
            return {"error": "Weather API timeout"}
        except Exception as e:
            print(f"Weather API error: {e}")
            return {"error": str(e)}

# Alternative weather API (for production, consider using a key-based service)
# async def get_weather_alternative(city: str, country: str = "Kenya") -> Optional[Dict]:
#     """Alternative weather API (requires API key)."""
#     api_key = os.getenv("WEATHER_API_KEY")
#     if not api_key:
#         return {"error": "Weather API key not configured"}
#
#     # Use your preferred weather API here
#     pass
```

Part 6: Document Enrichment Endpoint

Step 10: Manual Enrichment Trigger

```
# main.py (continued)

@app.post("/documents/{document_id}/enrich")
@limiter.limit("5/minute")
async def enrich_document(
    request: Request,
    document_id: int,
    current_user: User = Depends(get_current_manager),
    session: Session = Depends(get_session)
):
    """
    Manually trigger weather enrichment for a document.
    Useful for documents that failed initial enrichment.
```

```
    """
    document = session.get(Document, document_id)
    if not document:
        raise HTTPException(404, "Document not found")

    if document.status == "enriched":
        return {"message": "Document already enriched"}

    weather_data = await get_weather(document.city, document.country)
    if weather_data:
        document.weather_data = json.dumps(weather_data)
        document.weather_fetched_at = datetime.utcnow()
        document.status = "enriched"
        session.commit()
        return {
            "message": "Document enriched successfully",
            "weather": weather_data
        }
    else:
        document.status = "failed"
        session.commit()
        raise HTTPException(500, "Failed to enrich document with weather data")

@app.get("/documents/{document_id}/weather")
@limiter.limit("10/minute")
def get_document_weather(
    request: Request,
    document_id: int,
    current_user: User = Depends(get_current_user),
    session: Session = Depends(get_session)
):
    """Get the weather data associated with a document."""
    document = session.get(Document, document_id)
    if not document:
        raise HTTPException(404, "Document not found")

    # Staff can only view their own documents
    if current_user.role not in ["admin", "manager"] and document.uploader_id != current_user.id:
        raise HTTPException(403, "Access denied")

    if not document.weather_data:
        raise HTTPException(404, "No weather data available for this document")
```

```
    return {
        "document_id": document.id,
        "city": document.city,
        "country": document.country,
        "weather": json.loads(document.weather_data)
    }
```

Part 7: Exercises

Exercise 1: Document Search with Filters

Problem: Staff need to quickly find documents by city, date range, or status.

Task: Implement a search endpoint that supports multiple filters.

```
@app.get("/documents/search")
@limiter.limit("20/minute")
def search_documents(
    request: Request,
    q: Optional[str] = None,
    city: Optional[str] = None,
    status: Optional[str] = None,
    date_from: Optional[datetime] = None,
    date_to: Optional[datetime] = None,
    current_user: User = Depends(get_current_user),
    session: Session = Depends(get_session)
):
    """
    Search documents with multiple filters.
    """
    # Your code here...
    pass
```

Questions:

1. How would you make this search efficient with a large number of documents?
2. Should managers see all documents while staff see only their own? (Hint: Check the list endpoint)

Exercise 2: Document Versioning

Problem: When a document is re-uploaded with the same filename, the old version is lost.

Task: Implement document versioning where uploading a document with the same name creates a new version.

```
# Hint: Add a version field to the Document model
# version: int = Field(default=1)

# When uploading, check if a document with the same original_filename exists
```

If so, increment the version
Questions:

1. How would you track changes between versions?
2. Should you store the old version or delete it?

Exercise 3: Webhook Notification
Problem: Staff want to be notified when a document is successfully enriched.
Task: Implement a webhook endpoint that sends notifications to a configured URL when a document status changes.

```
@app.post("/webhooks/register")
def register_webhook(
    webhook_url: str,
    event_type: str,  # "document.enriched", "document.uploaded"
    current_user: User = Depends(get_current_admin),
    session: Session = Depends(get_session)
):
    """
    Register a webhook for document events.
    """
    # Your code here...
    pass
```

Questions:

1. How would you handle retries if the webhook fails?
2. What security measures would you put in place?

Submit as per the previous labs.